

(Multi-Agent) Reinforcement Learning: a primer, with applications in economics

Introduction and Basics of Reinforcement Learning

Aldo Glielmo

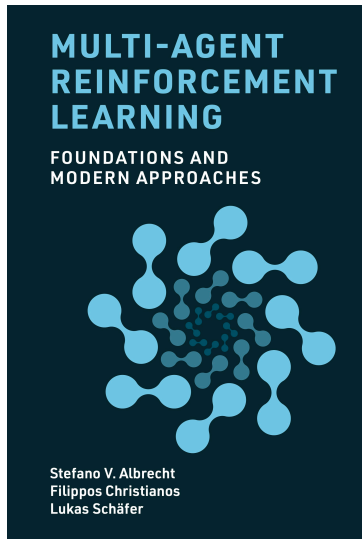
Applied Research Team, Banca d'Italia

A lot of material in this course is based on

Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

The book is freely available from:

www.marl-book.com



Part 1: Reinforcement Learning

- *L1: Intro and RL Basics* (Ch. 2)
- L2: Deep RL (Ch. 8, prerequisite: Ch. 7)

Part 2: Multi-Agent Reinforcement Learning

- L3: Games and MARL basics (Ch. 3-6)
- L4: Deep MARL (Ch. 9, 10)

Today's Lecture

Introduction

- Multi-agent systems and multi-agent RL

Reinforcement Learning Basics

- Markov decision processes
- Discounted returns and Bellman equations
 - ⇒ Short break (10 min, until around 11:30)
- Dynamic programming
- Temporal-difference learning
 - ⇒ Longer break (15 min, until around 12:15)

Interactive Session

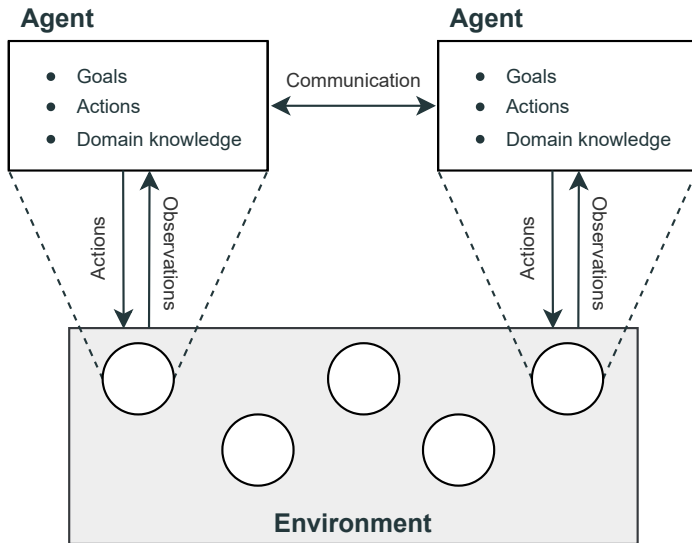
Introduction

A multi-agent system consists of:

- **Environment:** The environment is a physical or virtual world whose state evolves and is influenced by the agents' actions within the environment.
- **Agents:** An agent is an entity which receives information about the state of the environment and can choose actions to influence the state.

⇒ Agents are goal-directed, e.g. maximizing returns

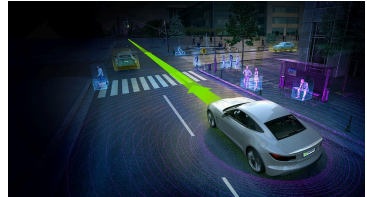
Multi-Agent Systems



Classic MAS Applications



Computer games



Autonomous driving



Multi-robot warehouses

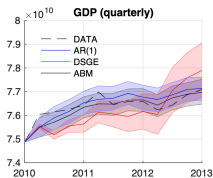


Automated trading

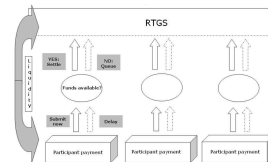
MAS Applications in Economics - Agent-Based Modelling



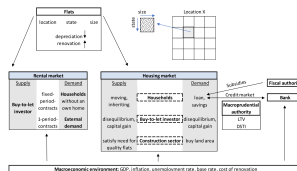
Stress testing



Economic forecasting



Payment systems simulation



Housing market analysis

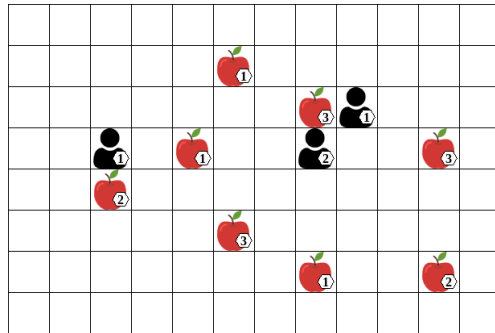
What is MARL?

Multi-agent reinforcement learning (MARL) is about finding optimal decision policies for two or more artificial agents interacting in a shared environment.

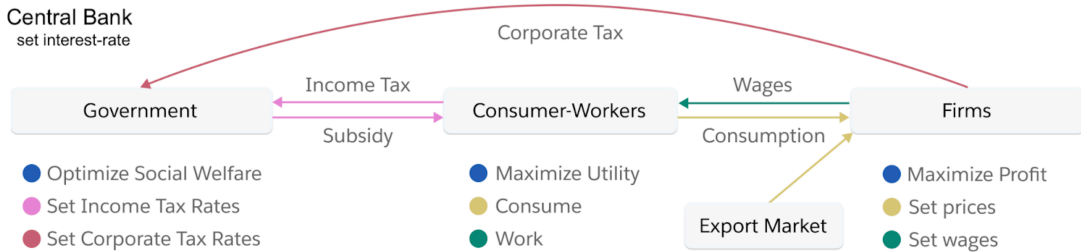
- Applying reinforcement learning (RL) algorithms to multi-agent systems
- Goal is to learn optimal policies for two or more agents

Example: Level-Based Foraging

- Three agents (robots) with varying skill levels
- Goal: to collect all items (apples)
- Items can be collected if a group of one or more agents are located next to the item and the sum of agents' levels is greater than or equal to the item level
- Action space
 $A = \{up, down, left, right, collect, noop\}$



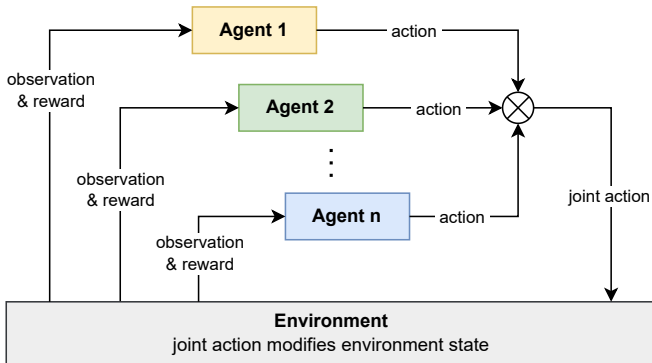
Example: The Economy as a System of Optimising Agents



[Curry et al., Analyzing Micro-Founded General Equilibrium Models ..., 2022]

- The economy can be viewed as a multi-agent system, where agents (e.g. consumers, firms, or the government) interact and make decisions to optimize their own objectives such as profits or general “utility”.

MARL for Solving Multi-Agent Systems



- **Goal:** learn optimal policies for a set of agents in a multi-agent system
- Each agent chooses an action based on its policy $\Rightarrow$ joint action
- Joint action affects environment state; agents get rewards + new observations

Why MARL?

Why should we use MARL to find optimal solutions to multi-agent systems rather than controlling multiple 'agents' using a single-agent RL (SARL) algorithm?

Decomposing a large problem

- For example, controlling 3 agents each with 6 actions, the joint action space becomes $6^3 = 216$.
⇒ Large action space for SARL!
- We can decompose this into three independent agents, each selecting from only 6 actions.
⇒ Use MARL to train separate agent policies (more tractable)

Decentralized decision making

- There are many real-world scenarios where it is required for each agent to make decisions independently.
- E.g. autonomous driving is impractical for frequent long-distance data exchanges between a central agent and the vehicle.

Reinforcement Learning Basics

What is Reinforcement Learning?

Reinforcement learning (RL)

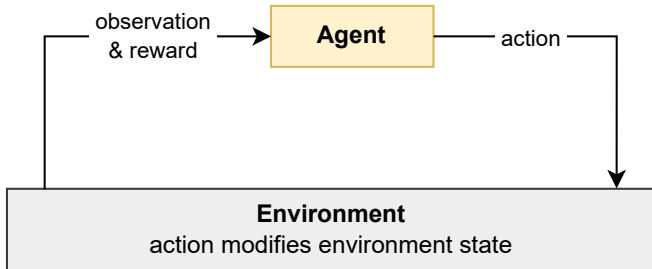
Learning to solve sequential decision problems via **repeated interaction with environment**

- What is a sequential decision problem?
- What does it mean to “solve” the problem?
- What is learning by interaction?

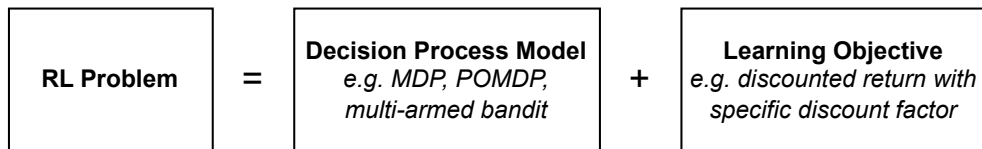
What is Reinforcement Learning?

Agent takes actions in environment

- Take action, observe new state and reward from environment
 - Goal is to maximise total rewards received
- ⇒ Learning: find best actions by *trying* them

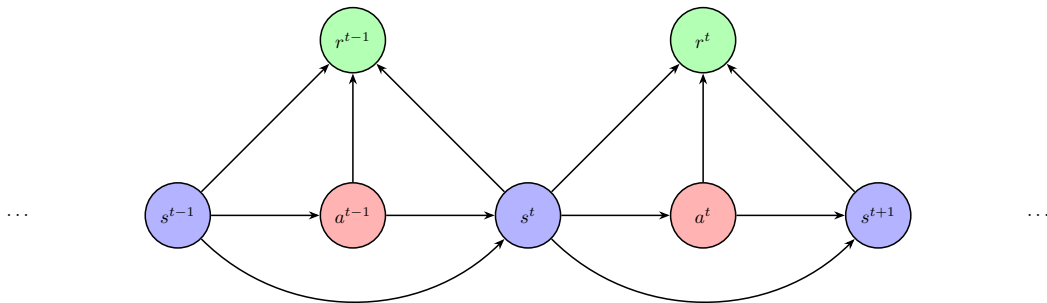


RL Problem Definition



- Sequential decision process is defined formally as **Markov decision process** (MDP)
- Goal is to learn an optimal decision policy for a specific learning objective

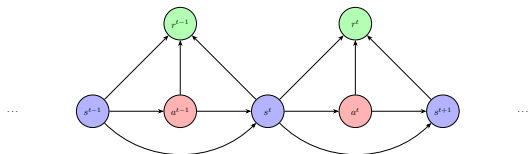
Markov Decision Process – Graph



Markov Decision Process – Definition

MDP is defined as:

- S : Finite set of states, with subset of terminal states $\bar{S} \subset S$
- A : Finite set of actions
- $\mathcal{R}$: Reward function $\mathcal{R} : S \times A \times S \rightarrow \mathbb{R}$
- $\mathcal{T}$: State transition function
 $\mathcal{T} : S \times A \times S \rightarrow [0, 1]$
- μ : Initial state distribution $\mu : S \rightarrow [0, 1]$



MDP Assumptions

Markov property

- Future states are temporally independent of past states and actions, given the current state and action.
- $Pr(s^{t+1}, r^t | s^t, a^t, s^{t-1}, a^{t-1}, \dots, s^0, a^0) = Pr(s^{t+1}, r^t | s^t, a^t)$

Full observability

- Agent can see the entire state of the environment.
- In many applications, agent may only have partial view.

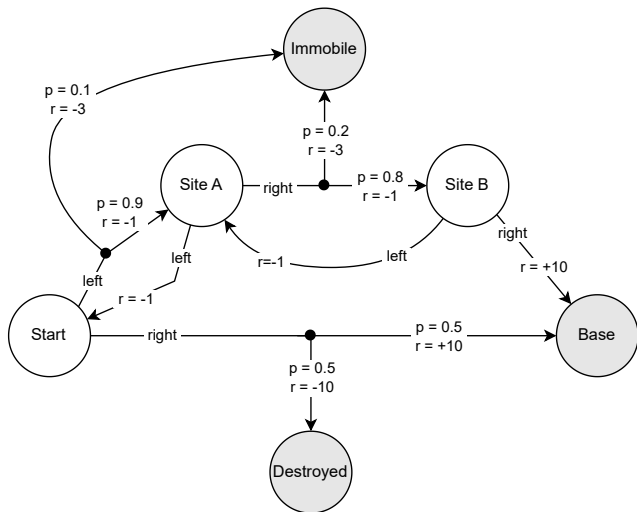
Stationarity

- Dynamics of the environment are assumed to be stationary.
- i.e. $\mathcal{T}$ and $\mathcal{R}$ are constant through time

Incomplete knowledge of MDP

- Agent may only have knowledge of the action and state spaces ($\mathcal{A}$, $\mathcal{S}$)
- Transition and reward functions ($\mathcal{T}$, $\mathcal{R}$) are usually assumed to be **unknown**.

Mars Rover MDP Example



- *Start* is the initial state s^0
- Two possible actions
 $A = \{right, left\}$
- Goal is to get to *Base* terminal state
- Rewards given by $\mathcal{R}(s, a, s')$ are shown as r
- State transition probabilities given by $\mathcal{T}(s, a, s')$, are shown as p

Expected Discounted Returns

Using RL, learn policy π that maximizes the expected **return**:

- Returns are the sum of rewards received over time
- If MDP is non-terminating (i.e. $t \rightarrow \infty$), we **discount** returns to ensure finite (discounted) returns

$$\mathbb{E}_{\pi}[u_t] = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r^t \right]$$

- γ is the discount factor, such that $\gamma \in [0, 1]$
- π is the policy that determines which actions are chosen

State-Value Functions

State-value functions $V^\pi(s)$ give the 'value' of state s when following the policy π :

$$V^\pi(s) = \mathbb{E}_\pi [u^t | s^t = s]$$

The return (u) can be recursively defined as:

$$\begin{aligned} u^t &= r^t + \gamma(r^{t+1} + \gamma r^{t+2} + \dots) \\ &= r^t + \gamma u^{t+1} \end{aligned}$$

Therefore:

$$V^\pi(s) = \mathbb{E}_\pi [r^t + \gamma u^{t+1} | s^t = s]$$

The Bellman Equation

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[r^t + \gamma u^{t+1} \mid s^t = s] \\ &= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) [\mathcal{R}(s, a, s') + \gamma \mathbb{E}_\pi[u^{t+1} \mid s^{t+1} = s']] \\ &= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \end{aligned}$$

The last equation is known as the **Bellman equation** in honor of Richard Bellman.

- The value of being in state s while following a fixed policy π is equivalent to the immediate reward ($\mathcal{R}(s, a, s') \rightarrow r$) received when taking action a in state s ($\pi(a \mid s)$) plus the discounted value of the next state s' .

State-Action Value Function

State-action value functions $Q^\pi(s, a)$ are an extension on the *State* value functions. They condition the expected return on the current state and the action taken:

$$Q^\pi(s, a) = \mathbb{E}_\pi [u^t | s^t = s, a^t = a]$$

The *state-action* value Bellman equation is therefore:

$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')]$$

Greedy Policies

A policy can act **greedily**: choosing actions that have maximum estimated value.

Greedy π using *state value* functions:

$$\pi(s) = \arg \max_{a \in A} \sum_{s', r} \mathcal{T}(s', r | s, a) [r + \gamma V(s')]$$

Or using the *state-action value function*:

$$\pi(s) = \arg \max_{a \in A} Q(s, a)$$

Optimal Greedy Policy

A greedy policy with respect to a value function is optimal only when using the **optimal value function**.

Optimal value function for a MDP is defined as:

$$V^*(s) = \max_{\pi'} V^{\pi'}(s), \quad \forall s \in S$$
$$Q^*(s, a) = \max_{\pi'} Q^{\pi'}(s, a), \quad \forall s \in S, a \in A$$

Therefore, the optimal policy is:

$$\pi^*(s) = \arg \max_{a \in A} Q^*(s, a)$$

Reinforcement Learning Basics

Dynamic Programming

Dynamic Programming

- Dynamic Programming (DP) is a family of algorithms to compute **optimal value functions** and **optimal policies in MDPs** (Bellman 1957; Howard 1960).
- In DP, we **assume complete knowledge** of the MDP, including the transition and reward function $(\mathcal{T}, \mathcal{R})$.
- Given complete knowledge, we can use the **Bellman equation** to find optimal value functions and policies.

Policy Iteration

Policy Iteration is a DP algorithm that alternates between two steps:

- **Policy evaluation:** compute value function V^π for current policy π
- **Policy improvement:** improve current policy π with respect to V^π

$$\pi^0 \rightarrow V^{\pi^0} \rightarrow \pi^1 \rightarrow V^{\pi^1} \rightarrow \pi^2 \rightarrow \dots \rightarrow V^* \rightarrow \pi^*$$

Policy Iteration Pseudo Code

Algorithm Policy Iteration

```
1: Initialize deterministic policy  $\pi$  randomly, and  $V(s) = 0$  for all  $s \in S$ 
2: repeat
3:   // Iterative Policy Evaluation:
4:   repeat
5:     for each state  $s$  in  $S$  do
6:        $V(s) \leftarrow \sum_{s'} \mathcal{T}(s'|s, \pi(s)) [\mathcal{R}(s, \pi(s), s') + \gamma V(s')]$ 
7:   until  $V(s)$  converges for all  $s \in S$ 
8:   // Policy Improvement:
9:   policy_stable  $\leftarrow$  true
10:  for each state  $s$  in  $S$  do
11:    old_action  $\leftarrow \pi(s)$ 
12:     $\pi(s) \leftarrow \arg \max_a \sum_{s'} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V(s')]$ 
13:    if old_action  $\neq \pi(s)$  then
14:      policy_stable  $\leftarrow$  false
15: until policy_stable return  $V, \pi$ 
```

Value Iteration uses the **Bellman optimality equation**

- Combines iterative policy evaluation and improvement into one update rule.

Bellman Optimality Equation as update operator:

$$V^{k+1}(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V^k(s')], \quad \forall s \in S$$

- The max-operator makes this the Bellman *optimality* equation.
- The equation expresses the value of a state as the maximum expected return achievable by taking the best action and then following the optimal policy.

Value Iteration Pseudo Code

Algorithm Value Iteration

- 1: Initialize: $V(s) = 0, \forall s \in S$
 - 2: **repeat**
 - 3: $\forall s \in S : V(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V(s')]$
 - 4: **until** V converged **return** optimal policy π^* with:
 - 5: $\forall s \in S : \arg \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s'|s, a) [\mathcal{R}(s, a, s') + \gamma V(s')]$
-

Dynamic Programming – Interactive Visualization

- Interactive visualization https://uonliaquat.github.io/RL_Visualizer/
- Source code https://github.com/uonliaquat/RL_Visualizer
- Some things to check:
 - ⇒ Can you identify precisely S , A , $\mathcal{T}$, $\mathcal{R}$ in the MDP?
 - ⇒ Can you understand the resulting value function and policy?
 - ⇒ Can you see the difference between policy iteration and value iteration?
 - ⇒ How does the value function change as discount γ or reward function $\mathcal{R}$ change?

Reinforcement Learning Basics

Temporal-Difference Learning

Temporal-Difference Learning

Temporal-Difference (TD) learning is a family of RL algorithms that learn optimal policies and value functions based on data collected via environment **interactions**.

- These algorithms learn which actions yield the best returns by **trial and error** and exploring different actions and states
 - ⇒ Learns without **model** of environment's **reward** and **transition** functions ($\mathcal{R}, \mathcal{T}$)
 - ⇒ Can learn **online**, updating the policy while interacting with the environment

Temporal-Difference Update

The update for temporal-difference learning uses value functions:

$$V(s^t) \leftarrow V(s^t) + \alpha [\mathcal{X} - V(s^t)]$$

or

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [\mathcal{X} - Q(s^t, a^t)]$$

- $\mathcal{X}$ is the update target, serving as an estimate of the current state value.
- α is the learning rate

Temporal-Difference Update

In SARSA (a basic TD algorithm), we use the experience tuple $(s^t, a^t, r^t, s^{t+1}, a^{t+1})$ to construct a target:

$$\mathcal{X} = r^t + \gamma Q(s^{t+1}, a^{t+1})$$

(The immediate reward plus the discounted value of the next state)

Note the resemblance to the Bellman equation:

$$Q^\pi(s, a) = \sum_{s' \in \mathcal{S}} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a' \in \mathcal{A}} \pi(a' \mid s') Q^\pi(s', a') \right]$$

The SARSA update rule thus becomes:

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha[r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t)]$$

- The TD update is **bootstrapped**: using the value estimates of the next state ($Q(s^{t+1}, a^{t+1})$) to update the current state value ($Q(s^t, a^t)$)
- $(r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t))$ is also called “TD error”

SARSA Pseudo Code

Algorithm SARSA

- 1: Initialize $Q(s, a) = 0$ for all $s \in S, a \in A$
 - 2: **for** every episode **do**
 - 3: Observe initial state s^0
 - 4: With probability ϵ : choose random action $a^0 \in A$
 - 5: Otherwise: choose action $a^0 \in \arg \max_a Q(s^0, a)$
 - 6: **for** $t = 0, 1, 2, \dots$ **do**
 - 7: Apply action a^t , observe reward r' and next state s^{t+1}
 - 8: With probability ϵ : choose random action $a^{t+1} \in A$
 - 9: Otherwise: choose action $a^{t+1} \in \arg \max_a Q(s^{t+1}, a)$
 - 10: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha[r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t)]$
-

Convergence of SARSA

SARSA is guaranteed to converge to the optimal **state-value function**, for all $S \in \mathcal{S}$ and $a \in A$, if:

- All *state-action* pairs are explored infinitely many times:

$$\forall s \in \mathcal{S}, a \in A : \sum_{k=0}^{\infty} \mathbb{I}(s, a) \rightarrow \infty$$

- The learning rate α is reduced over time according to the "standard stochastic approximation conditions":

$$\forall s \in \mathcal{S}, a \in A : \sum_{k=1}^{\infty} \alpha_k(s, a) \rightarrow \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \alpha_k(s, a)^2 < \infty$$

ϵ -Greedy Policies

Using a **greedy policy** would **violate the convergence condition** of SARSA (infinite exploration of S and A).

- Intuitively, we must explore a wide range of states and actions to find state action combinations that yield high returns
- One solution is to add an **exploration** parameter $\epsilon \in [0, 1]$. This gives us a stochastic **epsilon-greedy** policy

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A|} & \text{if } a \in \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|A|} & \text{otherwise} \end{cases}$$

- With probability $1 - \epsilon$, the policy chooses the greedy action, and with probability ϵ chooses an action uniformly at random

Q-learning (Watkins & Dayan 1992) is a popular TD algorithm which uses the Bellman optimality equation to update its value function estimates.

- By using the Bellman optimality equation, Q-learning directly learns the **optimal state-action value function**
- Q-learning is **off-policy**, meaning the policy followed to gather experiences is different from the optimized policy
- We use the ϵ -greedy policy to collect experiences

Q-Learning Update

The target in Q-learning uses the *max* operator to target the optimal Q-values directly:

$$\mathcal{X} = r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a')$$

The Q-learning update is thus:

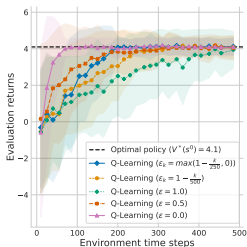
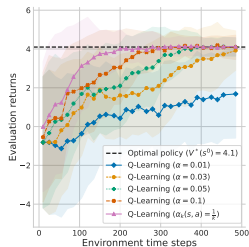
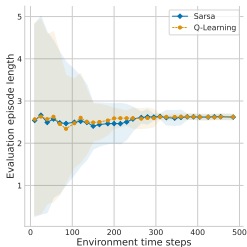
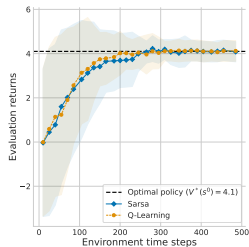
$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a') - Q(s^t, a^t) \right]$$

Q-Learning Pseudo Code

Algorithm Q-Learning

- 1: Initialize $Q(s, a) = 0$ for all $s \in S, a \in A$
 - 2: **for** every episode **do**
 - 3: **for** $t = 0, 1, 2, \dots$ **do**
 - 4: Observe current state s^t
 - 5: With probability ϵ : choose random action $a^t \in A$
 - 6: Otherwise: choose action $a^t \in \arg \max_a Q(s^t, a)$
 - 7: Apply action a^t , observe reward r^t and next state s^{t+1}
 - 8: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha [r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t)]$
-

Evaluating RL Algorithms



Y-axis:

- Average **discounted evaluation returns**.
This shows us how our greedy policy would perform if we stopped learning at time step T .
- In some cases, **undiscounted returns** are easier to interpret and may be used instead.

X-axis:

- Cumulative training steps across episodes.
- Number of episodes can also be used. This might, however, distort the learning speed.

Evaluating RL Algorithms Continued

Why Cumulative Time Steps and not n Episodes?

- Episodes can distort the speed at which an algorithm learns.
- Maybe, and the algorithm needs a few episodes to converge, but it takes a lot of time to explore in early episodes.

Average Discounted Evaluation Returns.

- Returns are from a greedy policy with respect to learned action values after T time steps.
- We average to answer the following question: if we finish learning after T time steps and extract the greedy policy, what expected returns can we expect to achieve with this policy?

What about Undiscounted Returns?

- We usually use discounted returns as this is the learning objective, but undiscounted rewards can sometimes be easier to interpret.

TD Learning – Interactive Visualization

- Interactive visualization <https://awjuliani.github.io/web-rl-playground/>
- Source code <https://github.com/awjuliani/web-rl-playground>
- Some things to check:
 - ⇒ Do you understand the exact learning dynamics of the algorithms?
 - ⇒ How does changing the learning rate α or the exploration rate ϵ affect learning?
 - ⇒ Can you see the difference between SARSA and Q-learning?
 - ⇒ What are the advantages and disadvantages of RL over DP?

We covered:

- Intro to multi-agent systems and MARL
- MDPs
- Value functions
- Dynamic programming (policy and value iteration)
- Temporal-difference learning (SARSA and Q-learning)

Next we'll cover:

- Deep Reinforcement Learning

Interactive Coding and Learning Session

References

- *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. See chapters 1 and 2. A possible future “bible” for multi-agent RL. [link](#)
- *Reinforcement Learning: An Introduction*. See chapters 1–6. The “bible” of single-agent RL. [link](#)

Computational Resources

- [marl-book code](#)
- [sarl-book code](#)

References

- *Advanced Macroeconomics*, David Romer.
- *PhD Macro Theory Notes*, Eric Sims. [link](#)

Computational Resources

- [quant-econ example \(Python\)](#)
- [quant-econ example \(Julia\)](#)

General References

- *Making Sense of Chaos: A Better Economics for a Better World.* [link to first pages](#)
- *Agent-based models in economics: A toolkit.* [link to first pages](#)
- *The Economy as an Evolving Complex System IV.* [link](#)
- *Agent-based modeling at central banks: recent developments and new challenges.* [link](#)

Multi-Agent Economics – Agent-Based Modelling. Continued

Macro-ABM References

- *Agent-Based Macroeconomics*. A nice review of well-known macro-ABMs. [link](#)
- *Tipping Points in Macroeconomic Agent-Based models*. The simplest macro-ABM, sometimes called “Mark0”/“Mark1”. [link](#)
- *Emergent Dynamics of a Macroeconomic Agent-Based Model with Capital and Credit*. A relatively simple macro-ABM, a kind of “Mark2”. [link](#)
- *Economic Forecasting with an Agent-Based Model*. Latest macro-ABM in the series (“Mark3”) and the first capable of quantitative forecasting. [link](#)

Computational Resources

- [Mark0.jl](#) (The “Mark0” above)
- [ABCredit.jl](#) (The “Mark2” above)
- [BeforeIT.jl](#) (The “Mark3” above)